

Agentic AI — WealthWise

 Anthropic Claude

 Sonnet 4

 Google Gemini

 Gemini 3

 MCP SDK

 1.12

 TypeScript

 5.5

 Express

 4

 Vitest

 2

 Docker

 Ready

 Podman

 Ready

An **agentic AI service** for WealthWise, powered by Anthropic's Claude. Four specialized financial agents consume MCP tools from the WealthWise MCP server to access real user financial data, analyze it through multi-step tool-use loops, and return actionable insights — all behind the same JWT authentication as the main API.

Table of Contents

- [Architecture](#)
- [Agents](#)
- [API Endpoints](#)
- [Request / Response Examples](#)
- [Tool Use Loop](#)
- [Conversation Management](#)
- [Cost Tracking](#)
- [Environment Variables](#)
- [Development](#)
- [Testing](#)

Architecture

```
graph TB
  subgraph Client["Frontend (Next.js)"]
    UI["React UI"]
  end

  subgraph AgenticAI["Agentic AI Service :5200"]
    direction TB
    REST["REST API  
Express 4"]
    AUTH["Auth Middleware  
JWT Verification"]
    RL["Rate Limiter"]
    ORCH["Orchestrator Agent"]

    subgraph Specialists["Specialist Agents"]
      FA["Financial Advisor"]
      AD["Anomaly Detector"]
      BO["Budget Optimizer"]
      FC["Forecaster"]
    end
  end
```

```

    CONV["Conversation Manager"]
    COST["Cost Tracker"]
end

subgraph Claude["Anthropic API"]
    LLM["Claude Sonnet 4"]
end

subgraph MCP["MCP Layer"]
    MCPC["MCP Client
SSE Transport"]
    MCPS["MCP Server"]
end

DB[(MongoDB 7)]

UI -- "POST /api/v1/agent/*" --> REST
REST --> AUTH --> RL --> ORCH
ORCH -- "intent classification" --> LLM
ORCH -- "delegates" --> Specialists
Specialists -- "tool_use loop" --> LLM
LLM -- "tool_use blocks" --> MCPC
MCPC -- "SSE" --> MCPS
MCPS --> DB
Specialists --- CONV
Specialists --- COST

style Client fill:#0f172a,stroke:#6366f1,color:#e2e8f0
style AgenticAI fill:#0f172a,stroke:#d97706,color:#e2e8f0
style Specialists fill:#1e293b,stroke:#d97706,color:#e2e8f0
style Claude fill:#0f172a,stroke:#f59e0b,color:#e2e8f0
style MCP fill:#0f172a,stroke:#10b981,color:#e2e8f0
style DB fill:#0f172a,stroke:#47a248,color:#e2e8f0

```

The service sits between the frontend and Anthropic's Claude API. User queries arrive via REST, pass through JWT authentication and rate limiting, then enter the **orchestrator** which classifies intent and delegates to the appropriate specialist agent. Each specialist executes a multi-step **tool_use loop** — calling MCP tools to fetch financial data, feeding results back to Claude, and iterating until a final text response is produced.

Agents

Agent	Role	Example Queries
Orchestrator	Routes incoming messages to the appropriate specialist based on intent classification	<i>(internal — not called directly)</i>
Financial Advisor	Comprehensive financial health assessment, personalized advice, goal recommendations	"How is my financial health?", "What should I prioritize paying off?"

Agent	Role	Example Queries
Anomaly Detector	Unusual spending pattern detection, duplicate transaction identification, category spending spikes	"Are there any unusual charges this month?", "Do I have duplicate subscriptions?"
Budget Optimizer	Budget utilization analysis, reallocation suggestions, savings opportunity identification	"Am I on track with my budgets?", "Where can I cut spending?"
Forecaster	3/6/12-month financial projections, trend analysis, goal achievement predictions	"Will I hit my savings goal by December?", "Project my spending for the next quarter"

All specialist agents extend **BaseAgent**, which implements the core `tool_use` loop. The orchestrator makes a lightweight classification call to Claude, parses the JSON routing decision, and delegates to the chosen specialist with the full conversation context and MCP tool set.

API Endpoints

All endpoints are prefixed with `/api/v1/agent` and require a valid JWT Bearer token.

Method	Path	Description	Rate Limit
POST	<code>/agent/chat</code>	Conversational AI chat with orchestrated routing	20 req/min
POST	<code>/agent/insights</code>	One-shot insights by type (<code>financial-health</code> , <code>anomalies</code> , <code>budget-review</code> , <code>forecast</code>)	10 req/min
GET	<code>/agent/insights/summary</code>	Quick financial summary from the advisor agent	10 req/min
DELETE	<code>/agent/conversations/:id</code>	Clear conversation history for the authenticated user	--
GET	<code>/health</code>	Health check (no auth required)	--

Request / Response Examples

Chat

```
POST /api/v1/agent/chat
Authorization: Bearer <jwt-token>

{
  "message": "How is my spending this month compared to last month?"
}
```

```
{
  "success": true,
  "data": {
    "response": "Based on your transaction data, your total spending this month is $3,240 compared to $2,890 last month – a 12% increase. The biggest jump is in the Dining category, which went from $320 to $510. Your Groceries and Housing costs remained stable. I'd recommend keeping an eye on discretionary dining expenses to stay within your monthly budget.",
    "agent": "financial-advisor",
    "conversationId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "usage": {
      "inputTokens": 1250,
      "outputTokens": 480
    }
  }
}
```

Insights

```
POST /api/v1/agent/insights
Authorization: Bearer <jwt-token>
```

```
{
  "type": "anomalies"
}
```

```
{
  "success": true,
  "data": {
    "response": "I found 2 potential anomalies in your recent transactions...",
    "agent": "anomaly-detector",
    "conversationId": null,
    "usage": {
      "inputTokens": 980,
      "outputTokens": 320
    }
  }
}
```

Error

```
{
  "success": false,
  "error": {
    "code": "RATE_LIMITED",
  }
}
```

```

    "message": "Too many chat requests. Limit: 20 per minute."
  }
}

```

Tool Use Loop

```

sequenceDiagram
    participant User
    participant REST as REST API
    participant Orch as Orchestrator
    participant Agent as Specialist Agent
    participant Claude as Claude API
    participant MCP as MCP Client
    participant Server as MCP Server
    participant DB as MongoDB

    User->>REST: POST /agent/chat { message }
    REST->>Orch: route(message)
    Orch->>Claude: Classification call (system prompt + message)
    Claude-->>Orch: { agent: "financial-advisor", reason: "..." }
    Orch->>Agent: run(message, tools, mcpClient, history)

    loop Tool Use Loop (max 15 iterations)
        Agent->>Claude: messages + MCP tools
        Claude-->>Agent: tool_use block (e.g., get_transactions)
        Agent->>MCP: callTool("get_transactions", { period: "month" })
        MCP->>Server: SSE request
        Server->>DB: Query
        DB-->>Server: Results
        Server-->>MCP: Tool result
        MCP-->>Agent: JSON result
        Agent->>Claude: tool_result + continue
    end

    Claude-->>Agent: Final text response (stop_reason: end_turn)
    Agent-->>Orch: AgentResponse
    Orch-->>REST: { response, agent, usage }
    REST-->>User: JSON response

```

Each iteration of the loop:

1. The agent sends the full message history (including prior tool results) to Claude with the available MCP tools.
2. Claude returns either a **tool_use** block (requesting data) or a **text** block (final answer).
3. If **tool_use**, the agent executes the tool via the MCP client, appends the result, and loops.
4. If **text** or max iterations (15) reached, the loop exits and the response is returned.

Tool execution errors are caught and fed back to Claude as error results, allowing the model to recover or provide a partial answer.

Conversation Management

- **Per-user state:** Each authenticated user gets a single conversation stored in memory, keyed by `userId`.
- **Multi-turn context:** Message history (user + assistant turns) is passed to the agent on every request, enabling follow-up questions.
- **Auto-expiry:** Conversations are cleaned up after 30 minutes of inactivity (configurable via `ConversationManager` TTL).
- **Cleanup interval:** A background timer runs every 5 minutes to evict expired conversations.
- **Manual clear:** Users can call `DELETE /agent/conversations/:id` to reset their conversation.
- **Graceful shutdown:** The cleanup interval is stopped on `SIGINT/SIGTERM`.

Cost Tracking

The service tracks Claude API usage per user:

Metric	Description
<code>inputTokens</code>	Cumulative input tokens across all requests
<code>outputTokens</code>	Cumulative output tokens across all requests
<code>requests</code>	Total number of agent invocations
<code>lastModel</code>	Most recently used model (<code>claude-sonnet-4-20250514</code>)
<code>lastUpdated</code>	Timestamp of the last request

Usage is tracked in-memory and can be queried or reset programmatically. This data supports monitoring dashboards and billing awareness.

Environment Variables

Variable	Required	Default	Description
<code>ANTHROPIC_API_KEY</code>	Yes	--	Anthropic API key for Claude access
<code>MCP_SERVER_URL</code>	Yes	--	URL of the WealthWise MCP server (e.g., <code>http://localhost:5100/sse</code>)
<code>JWT_SECRET</code>	Yes	--	JWT secret (must match the main API, min 32 chars)
<code>AGENT_PORT</code>	No	<code>5200</code>	Port the agentic AI service listens on
<code>NODE_ENV</code>	No	<code>development</code>	<code>development</code> , <code>production</code> , or <code>test</code>

Development

```
# Start in dev mode (requires MCP server running)
npm run dev -w agentic-ai

# Run tests (31 tests)
npx turbo test --filter=@wealthwise/agentic-ai

# Build
npx turbo build --filter=@wealthwise/agentic-ai

# Type check
npx turbo lint --filter=@wealthwise/agentic-ai

# Run with Docker
docker build -f agentic-ai/Dockerfile -t wealthwise-agentic-ai .
docker run -p 5200:5200 --env-file .env wealthwise-agentic-ai

# Run with Podman
podman build -f agentic-ai/Containerfile -t wealthwise-agentic-ai .
podman run -p 5200:5200 --env-file .env wealthwise-agentic-ai
```

[!NOTE] The MCP server must be running and accessible at `MCP_SERVER_URL` before starting the agentic AI service. The MCP server provides the financial data tools that agents use to access MongoDB.

Testing

- **31 tests** using Vitest
- Test files in `agentic-ai/src/__tests__`

Test Suite	Coverage
<code>agents/orchestrator.test.ts</code>	Routing logic, fallback to default agent, error handling
<code>agents/financial-advisor.test.ts</code>	Tool use loop, system prompt loading, response formatting
<code>agents/anomaly-detector.test.ts</code>	Anomaly detection flow, tool execution
<code>agents/budget-optimizer.test.ts</code>	Budget analysis flow, tool execution
<code>agents/forecaster.test.ts</code>	Forecasting flow, tool execution
<code>mcp/client.test.ts</code>	MCP client creation, tool listing, SSE transport
<code>routes/agent.routes.test.ts</code>	Endpoint validation, rate limiting, auth, error responses

The Anthropic SDK and MCP client are fully mocked in tests — no external API calls are made during test execution.

```
# Run with coverage
npx turbo test:coverage --filter=@wealthwise/agentic-ai
```

```
# Watch mode for development  
npm run test:watch -w agentic-ai
```